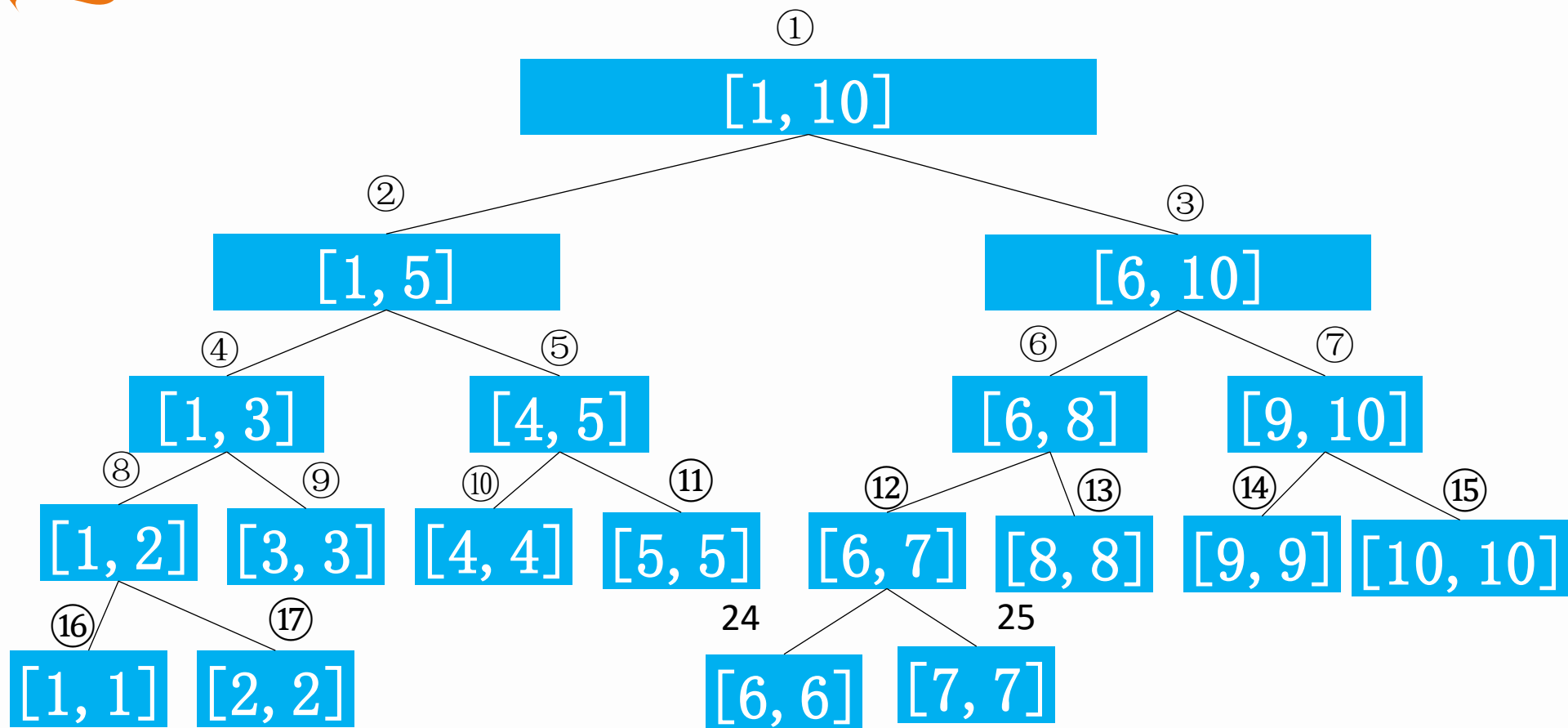


线段树的形态



线段树的建树

```
void build(int id,int l,int r)
{
    tree[id].l=l;
    tree[id].r=r;
    if(l==r)//递归终止条件
    {
        tree[id].max=a[l];
        tree[id].sum=a[l];
        return ;
    }
    int mid=(l+r)/2;
    build(id*2,l,mid);//编号为id的节点的儿子是固定的2*id和2*id+1
    build(id*2+1,mid+1,r);
    tree[id].max=max(tree[id*2].max,tree[id*2+1].max);
    tree[id].sum=tree[id*2].sum+tree[id*2+1].sum;
    //递归回去的时候更新父亲节点的max和sum值
    return ;
}
```

单点更新

```
void update(int id,int k,int x)
{
    if(tree[id].l==tree[id].r)
    {
        tree[id].max=x;
        tree[id].sum=x;
        return ;
    }
    int mid=(tree[id].l+tree[id].r)/2;
    if(k<=mid)//在左区间就更新左区间
        update(id*2,k,x);
    else//在右区间就更新右区间
        update(id*2+1,k,x);
    tree[id].max=max(tree[id*2].max,tree[id*2+1].max);
    tree[id].sum=tree[id*2].sum+tree[id*2+1].sum;
    //子节点信息被修改，其父亲节点信息也要更新
}
```

区间查询

```
void query(int id,int l,int r)
{
    if(tree[id].l==l&&tree[id].r==r)// 找到一个完整区间
    {
        return tree[id].sum;
    }
    int mid=(tree[id].l+tree[id].r)/2;
    if(r<=mid)// 都在左区间
        return query(id*2,l,r);
    else if(l>mid)// 都在右区间
        return query(id*2+1,l,r);
    else// 左右子树都有
        return query(id*2,l,mid)+query(id*2+1,mid+1,r);
}
```

区间更新

```
void update(int id,int l,int r,int x)
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        // 修改懒标记
        // 计算区间真实和
        return ;
    }
    // 如果在下一层, 先标记下传, 再遍历儿子
    if(tree[id].lazy)
        pushdown(id);
    int mid=(tree[id].l+tree[id].r)/2;
    if(r<=mid)// 和区间查询几乎一致
        update(id*2,l,r,x);
    else
        update(id*2+1,l,r,x);
    else
    {
        update(id*2,l,mid);
        update(id*2+1,mid+1,r);
    }
    tree[id].sum=tree[id*2].sum+tree[id*2+1].sum;
}
```

懒标记下传

```
void down(int id)
{
    tree[id*2].lazy+=tree[id].lazy;
    tree[id*2+1].lazy+=tree[id].lazy; // 先下传父亲节点懒标记
    tree[id*2].sum+=(tree[id*2].r-tree[id*2].l+1)*tree[id].lazy;
    tree[id*2+1].sum+=(tree[id*2+1].r-tree[id*2+1].l+1)*tree[id].lazy;
    // 同时还要更新区间之和，注意是*新加的懒标记而不是总共的懒标记
    tree[id].lazy=0; // 懒标记清零
}
```

区间更新时要注意懒标记下传的位置，不能再区间判等之前，因为这样会导致访问到叶子节点的下下一个节点，所以会导致RE，如果放在更新左右区间之后肯定就错误了。

区间查询

```
void query(int id,int l,int r)
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        return tree[id].sum;
    }
    if(tree[id].lazy)
        pushdown(id);
    int mid=(l+r)/2;
    if(r<=mid)
        return query(id*2,l,r);
    else
        return query(id*2+1,l,r);
    else
        return query(id*2+1,l,mid)+query(id*2+1,mid+1,r);
}
```

动态开点线段树

线段树需要开4倍空间，所以有的时候比较耗费空间，因为他的编号并不是连续的，所以实际上线段树也是可以动态开点的，方法和01trie比较类似，新建一个节点对这个节点开一个编号。

但是这样编号为 id 的节点的左右儿子就不是 $id*2$ 和 $id*2+1$ 了，所以我们还在要在结构体内加两个变量分别记录左右儿子的编号。

总体来说，动态开点空间比常规线段树少一半，甚至1/4都可以。而且对于数据很大，但是操作很少的题，这是常规线段树解决不了的。

动态开点线段树

```
int build(int l,int r)
{
    ++k;//k表示当前节点编号
    if(l==r)
    {
        tree[id].sum=a[l];
        return k;//把当前节点编号返回回去
    }
    int mid=(l+r)/2;
    tree[k].lson=build(l,mid);
    tree[k].rson=build(mid+1,r);
    tree[k].sum=tree[tree[k].lson].sum+tree[tree[k].rson].sum;
    return k;//把节点编号返回去
}
root=build(1,n);
```



标记永久化

线段树区间修改区间查询有两种实现方式，一种是懒标记下传，另一种是标记永久化，这两种实现方式的效率都是一样的，一半情况下，普通线段树中常用懒标记下传，而二维线段树(树套树)中只能用标记永久化，而不能用懒标记下传的方式。





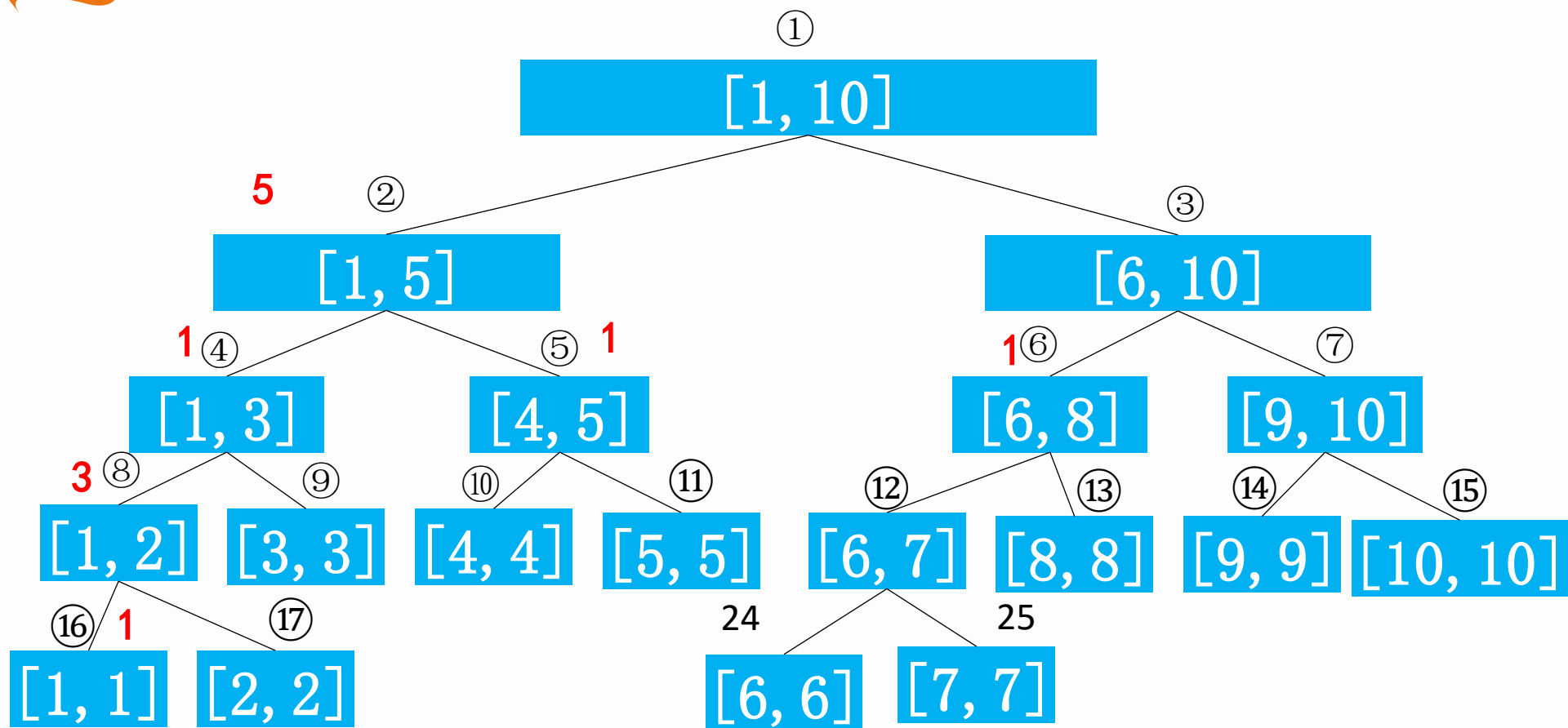
标记永久化

标记永久化的实现特别简单。

首先是更新的时候，遇到标记不下传，还是向下找刚好合适的区间加标记，查询的时候我们从根节点一层一层的找到目标区间所在的线段树的层数。在查找的过程中经过了多少标记那么就累加标记。



线段树的形态



如果要查询区间 $[1, 1]$ ，那么只需要知道从根节点到区间 $[1, 1]$ 经过的标记之和是多少 $= 10$

标记永久化

```
void query(int id,int l,int r,int s)//s表示路径上的标记之和
{
    if(tree[id].l==l&&tree[id].r==r)
    {
        return tree[id].sum+s*(tree[id].r-tree[id].l+1);
    }
    int mid=(l+r)/2;
    if(r<=mid)
        return query(id*2,l,r,s+tree[id].lazy);
    else
        return query(id*2+1,l,r,s+tree[id].lazy);
    else
        return query(id*2+1,l,mid,s+tree[id].lazy)+query(id*2+1,mid+1,r,s+tree[id].lazy);
}
```

练习题

```
题目.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
收作业(homework.cpp)
【问题描述】
xlj是一名来自00001110班的妹子，她是英语科代表。
xlj回到了教室，可怜的她要收英语作业，但是00001110班这群不负责任的组长把作业收得乱七八糟，散
得每个座位上都有作业本，xlj只好挨个去收。
00001110班的教室可以看作是一个n行m列的矩形，xlj在(0,0)这个格子（位于教室的左下角），教室的
门在(n-1,m-1)这个格子。每次xlj可以向相邻的格子走一步，走到某个格子时，她会收完这个格子的英语
作业。xlj还有许多事要做，她想走一条最短路到教室的门口，但是作业收得太少了又会引起公愤，所以她
决定走一条作业收得最多的最短路线(即保证路径最短的前提下收的作业最多)，你能帮帮她吗？
【输入格式】
第一行三个整数n,m,k。
接下来k行，每行三个整数a,b,c，表示在(a,b)这个格子有c本英语作业。
【输出格式】
输出1个整数，表示xlj最多能够收到的英语作业本数。
【输入样例】
2 2 2
0 0 1
1 1 1
【输出样例】
2
【样例解释】
(0,0)→(0,1)→(1,1) 共收作业2本
【数据范围】
对于30%的数据  $m \leq 30, k \leq 100$ 
```

第 1 行, 第 1 列 100% Windows (CRLF) ANSI